

Manual de Desarrollador Gestor de Identidades de Casa Monarca

Versión 1.0

Equipo de trabajo

Nombre	Matrícula
Ana Lidia Arteaga Bretón	A00838732
Ricardo Ballesteros Amavizca	A01255358
Daniel Garnelo Martínez	A00573086
Daniela Ortiz Blanco	A00840140
Axel Gabriel Rocha Pérez	A01067854
Álvaro Yeudiel Padilla Lobatón	A01665850

Profesores

Alberto F. Martínez
Raúl Gómez Muñoz
Daniel Otero Fadul
Pedro Leonel Olaya Trejos

Socio Formador

Casa Monarca

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas

Uso de álgebras modernas para seguridad y criptografía (Gpo 601)

Monterrey, Nuevo León. 12 de junio de 2026

Índice general

1. Descripción General	1
1.1. Propósito del sistema	1
1.2. Alcance	1
1.3. Casos de uso principales	2
2. Arquitectura del Sistema	3
2.1. Tipo de arquitectura	3
2.2. Componentes principales	3
2.3. Diagrama general	4
3. Stack Tecnológico	5
3.1. Lenguajes	5
3.2. Frameworks	5
3.3. Bibliotecas	5
4. Estructura del Proyecto	6
4.1. Organización de carpetas	6
4.2. Descripción de módulos	6
5. Configuración del Entorno	8
5.1. Requisitos (versiones)	8
5.2. Variables de entorno	8
5.3. Archivos de configuración	8
6. Instalación y Ejecución	10
6.1. Pasos para levantar el sistema	10
6.2. Comandos principales	10
7. Base de Datos	12

7.1. Modelo de datos	12
7.2. Tablas principales	12
7.3. Relaciones	13
8. API / Interfaces	14
8.1. Endpoints	14
8.2. Métodos y parámetros	14
8.3. Ejemplos de uso	14
9. Flujos y Lógica de Negocio	16
9.1. Procesos clave del sistema	16
9.2. Reglas importantes	17
9.3. Casos críticos	17
10. Ejemplos de Uso	18
10.1. Casos típicos de uso	18
10.2. Integración básica	18
10.3. Ejecución de funciones clave	18
11. Manejo de Errores	19
11.1. Estrategia de errores	19
11.2. Mensajes comunes	19
11.3. Registro (logs)	19
12. Pruebas	20
12.1. Tipos de tests	20
12.2. Cómo ejecutarlos	20
13. Despliegue	21
13.1. Cómo hacerlo	21
13.2. Entornos	21
13.3. Consideraciones	22
14. Seguridad	23
14.1. Autenticación	23
14.2. Manejo de datos sensibles	23
14.3. Buenas prácticas	23

15. Guía de Contribución	24
15.1. Convenciones de código	24
15.2. Flujo de trabajo (git)	24
15.3. Cómo agregar funcionalidades	24
16. Problemas Conocidos	25
16.1. Bugs actuales	25
16.2. Limitaciones	25
17. FAQ Técnica	26
17.1. Problemas comunes al levantar el proyecto	26
17.2. Errores típicos de desarrollo	26
18. Referencias	27

Capítulo 1

Descripción General

1.1. Propósito del sistema

El **Gestor de Identidades de Casa Monarca** es una aplicación web cuyo propósito es administrar, de forma segura y trazable, las identidades digitales y los privilegios de acceso del personal de Casa Monarca, así como proteger la información sensible de los expedientes de las personas migrantes y refugiadas que la organización atiende. Aunque la aplicación también digitaliza el levantamiento de expedientes (denominados *encuestas* en el código), su columna vertebral es un *stack* de gestión de identidad y seguridad: autenticación robusta de contraseñas, control de acceso basado en roles (RBAC), infraestructura de clave pública (PKI X.509), llaves de acceso WebAuthn (*passkeys*), cifrado de campo con rotación de llaves y una bitácora de auditoría.

En otras palabras, el sistema responde a una necesidad concreta del Socio Formador: una organización con recursos limitados que custodia datos personales altamente delicados —cuya filtración podría poner en riesgo físico y legal a personas ya vulnerables— requiere mecanismos de identidad y protección de datos del nivel de una institución grande, pero operables y mantenibles con su propia infraestructura. El gestor de identidades provee exactamente esa capa: decide *quién* puede entrar, *qué* puede hacer según su rol, *cómo* se prueba criptográficamente su identidad para las acciones críticas, y *queda registro* de cada operación relevante.

1.2. Alcance

Qué hace

- **Autenticación de usuarios** con hashing **Argon2id**, bloqueo de cuentas por intentos fallidos y verificación de contraseñas filtradas contra la base de datos de *Have I Been Pwned* (HIBP) mediante k-anonimato.
- **Control de acceso basado en roles** (RBAC) con cuatro roles (usuario, operativo, coordinador, admin) y un conjunto de permisos CRUD por rol.
- **Identidad reforzada para roles críticos** (admin, coordinador) vía certificados **X.509** con reto–respuesta firmado (RSA PKCS#1 v1.5 + SHA-256) y **passkeys WebAuthn**, con derivación automática de un

certificado a partir de la primera passkey (arquitectura N:1).

- **Cifrado de campo** (Fernet) de los datos sensibles de cada expediente, con **rotación de llaves** basada en un *keyring* con huellas SHA-256 y re-cifrado masivo en segundo plano.
- **Flujo de expedientes** con una máquina de estados revisada en cascada por los distintos niveles operativos.
- **Solicitudes ARCO** (Acceso, Rectificación, Cancelación y Oposición) con un pipeline de aprobación multinivel.
- **Auditoría** (tabla `logs`) y **respaldos cifrados** de la base de datos.

Qué límites tiene

- No expone una API JSON/REST de propósito general: la interacción es por formularios HTML renderizados en el servidor (Jinja2). Solo los flujos WebAuthn y algunos paneles de administración responden JSON consumido por `fetch()`.
- No integra directorios corporativos (LDAP/Active Directory) ni inicio de sesión único (SSO); las cuentas se administran localmente.
- No usa un módulo de seguridad por hardware (HSM): el material criptográfico reside en archivos locales protegidos (`key.key`, `certs/`, `keys/`).
- No está diseñado para escalado horizontal con estado compartido: usa SQLite como un único proceso. Esto condiciona el despliegue (ver el capítulo de Despliegue y el de Problemas Conocidos).
- El segundo factor para roles críticos es exclusivamente certificado X.509 + passkey; no hay 2FA por SMS o correo.

1.3. Casos de uso principales

1. **Inicio de sesión seguro.** Un colaborador se autentica con usuario y contraseña; si su rol es crítico, el sistema le exige completar y usar su certificado/passkey antes de operar.
2. **Levantamiento y canalización de un expediente.** Un usuario crea un expediente (estado borrador); este avanza por revisión de operativo, coordinador y, finalmente, `admin`, cifrando los datos sensibles en cada paso.
3. **Administración de identidades.** Un `admin` emite, descarga o revoca certificados, gestiona passkeys y consulta el estado del vínculo passkey–certificado.
4. **Rotación de llave de cifrado.** Un `admin`, autorizando la acción con su passkey, configura y activa una nueva llave; el sistema re-cifra los expedientes en segundo plano.
5. **Solicitud ARCO.** Una persona presenta una solicitud sobre sus datos en un formulario público; la solicitud escala por niveles hasta su resolución o reenvío a `admin`.
6. **Auditoría.** Un `admin` revisa la bitácora de eventos de seguridad y operación.

Capítulo 2

Arquitectura del Sistema

2.1. Tipo de arquitectura

El sistema sigue una arquitectura **monólito web cliente-servidor** renderizada en el servidor (*server-side rendering*). Es una aplicación Flask de *archivo único*: casi toda la lógica de negocio, el acceso a datos, los *helpers* criptográficos y las rutas viven en `app.py` (alrededor de 5 600 líneas, unas 40 rutas). No hay una capa de modelos u ORM separada: el acceso a datos se hace con `sqlite3` crudo a través de `get_conn()`. La configuración por ambiente se centraliza en `config.py`.

Este estilo se eligió deliberadamente por **simplicidad operativa**: un solo proceso, una sola base de datos en archivo, sin dependencias de infraestructura pesada, de modo que el Socio Formador pueda instalarlo y mantenerlo con recursos modestos.

2.2. Componentes principales

- **Capa web (Flask + Jinja2)**: define las rutas `@app.route`, procesa formularios y renderiza plantillas HTML en `templates/`.
- **Control de acceso (RBAC)**: `PERMISSIONS`, `has_permission()`, `require_role()` y el *gating* de identidad `enforce_certificate_setup()`.
- **Subsistema de identidad reforzada**: PKI X.509 (CA local en `certs/`) y WebAuthn/passkeys (`passkey_credentials`).
- **Subsistema de cifrado**: Fernet con *keyring* de llaves (`encrypt_data/decrypt_data`), métricas y cola de re-cifrado.
- **Persistencia**: base de datos SQLite (`database.db`) con 11 tablas, creada y migrada por `init_db()` mediante `ensure_column()`.
- **Auditoría**: función `log()` que escribe en la tabla `logs`.
- **Procesos auxiliares** (`tools/`): respaldo/restauración cifrados y *worker* de re-cifrado que consume la cola `reencrypt_jobs`.

2.3. Diagrama general

La Figura 2.1 muestra la relación entre los componentes: el navegador interactúa por HTTP con `app.py`, que concentra la lógica y persiste en SQLite; los procesos auxiliares operan fuera del ciclo de petición/respuesta sobre la misma base de datos y el material de llaves.

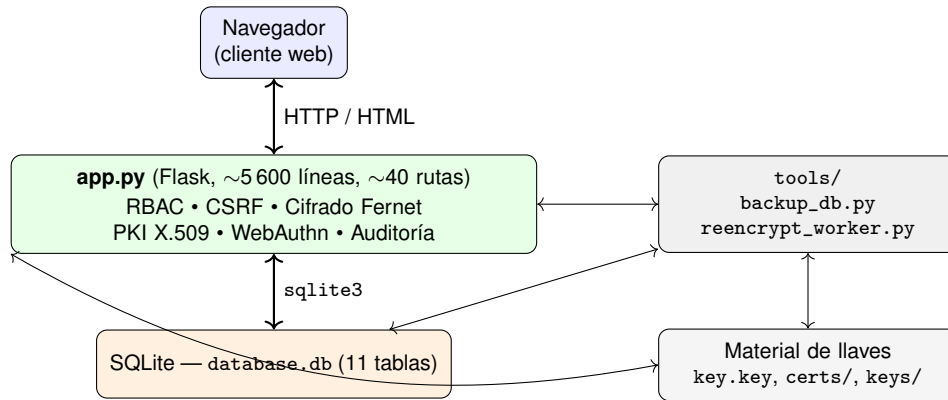


Figura 2.1: Diagrama general de componentes del Gestor de Identidades.

Capítulo 3

Stack Tecnológico

3.1. Lenguajes

El backend está escrito íntegramente en **Python** (3.10 o superior). Las plantillas usan **HTML** con el lenguaje de plantillas **Jinja2**, y el lado del cliente emplea **JavaScript** estándar del navegador para las ceremonias WebAuthn (`navigator.credentials.create/get`) y algunas llamadas `fetch()`. Las consultas a la base de datos se expresan en **SQL** (dialecto SQLite).

3.2. Frameworks

- **Flask 2.3.3** — framework web: enrutamiento, sesiones, render de plantillas.
- **Werkzeug 2.3.7** — capa WSGI subyacente de Flask.
- **Jinja2 3.1.2** — motor de plantillas HTML.

3.3. Bibliotecas

La Tabla 3.1 resume las dependencias declaradas en `requirements.txt` y su función criptográfica o de soporte.

Biblioteca	Versión	Uso
<code>cryptography</code>	$\geq 43.0.3, < 46$	Fernet, X.509, RSA, <i>padding</i> PKCS1v15, hashes SHA-256.
<code>argon2-cffi</code>	23.1.0	Hashing de contraseñas Argon2id.
<code>webauthn</code>	2.3.0	Ceremonias WebAuthn (registro y autenticación de passkeys).
<code>requests</code>	2.31.0	Llamadas HTTP salientes auxiliares.
<code>pytest / pytest-cov</code>	7.4.3 / 4.1.0	Suite de pruebas y cobertura.

Cuadro 3.1: Bibliotecas principales del proyecto.

La base de datos es **SQLite**, incluida en la biblioteca estándar de Python (`sqlite3`), por lo que no requiere un servidor de base de datos aparte.

Capítulo 4

Estructura del Proyecto

4.1. Organización de carpetas

```
casa-monarca-app/
|- app.py           # Backend completo: rutas, esquema DB, criptografia, reglas
|- config.py        # DevelopmentConfig / ProductionConfig / TestingConfig
|- generate_key.py   # Genera la llave Fernet maestra (key.key)
|- requirements.txt
|- setup.sh         # Instalacion: .venv, deps, .env, key.key, carpetas
|- .env.example     # Plantilla de variables de entorno
|- .gitignore       # Excluye key.key, keys/, certs/*.pem, *.db, etc.
|- templates/       # 14 plantillas Jinja2 (HTML)
|- static/          # Activos estaticos (CSS/JS/imagenes)
|- tools/
|   |- backup_db.py  # Respaldo cifrado de database.db (Fernet)
|   |- restore_db.py # Restauracion desde respaldo cifrado
|   |- reencrypt_worker.py # Procesa la cola reencrypt_jobs (rotacion)
|- tests/           # Suite pytest (fixtures con DB/certs/llaves aislados)
|- certs/           # CA local + certificados emitidos (gitignored)
|- keys/            # Material de llaves rotadas (gitignored)
|- backups/         # Respaldos cifrados
|- logs/            # Logs de ejecucion
|- key.key          # Llave Fernet activa (gitignored)
|- database.db      # Base de datos SQLite (gitignored, creada al arrancar)
```

4.2. Descripción de módulos

- `app.py` — núcleo de la aplicación. Para orientarse conviene buscar `@app.route` (rutas) y los `def` de nivel superior (helpers). Agrupa: esquema y migraciones (`init_db`, `ensure_column`), RBAC, contraseñas (Argon2id, política, HIBP), PKI X.509, WebAuthn/passkeys, cifrado Fernet, CSRF y banderas de cookies, y la lógica de expedientes y ARCO.
- `config.py` — jerarquía `Config` → `DevelopmentConfig`/`ProductionConfig`/`TestingConfig`, seleccionada con la variable `FLASK_ENV`.
- `generate_key.py` — genera la llave maestra Fernet en `key.key`.

- `tools/` — utilidades fuera del ciclo de petición: respaldo y restauración cifrados y el *worker* de re-cifrado.
- `templates/` — 14 plantillas Jinja2 (login, dashboard, survey, admin, admin_cifrado, admin_identidades, admin_pki, arco, cert_setup, colaborador, logs, password_update, profile, usuarios).
- `tests/` — pruebas `pytest` con *fixtures* que reconstruyen una base de datos, certificados y llaves aislados por prueba.

Capítulo 5

Configuración del Entorno

5.1. Requisitos (versiones)

- **Python** 3.10 o superior.
- Las dependencias de `requirements.txt` (ver Tabla 3.1).
- Sistema operativo tipo Unix (macOS o Linux) para `setup.sh`; en Windows pueden ejecutarse los pasos manualmente dentro de un entorno virtual.

5.2. Variables de entorno

La configuración se controla con variables de entorno (plantilla en `.env.example`). Las más relevantes, con sus valores por defecto, se listan en la Tabla 5.1.

5.3. Archivos de configuración

- `.env` — valores del entorno local (copiado de `.env.example` por `setup.sh`); está en `.gitignore`.
- `config.py` — define las tres clases de configuración. En `TestingConfig`, `LOGIN_MAX_ATTEMPTS` sube a 1000 para no interferir con las pruebas; `ProductionConfig` fuerza `SESSION_COOKIE_SAMESITE="Strict"` y desactiva `DEBUG`.
- `key.key`, `certs/*.pem`, `keys/` — material criptográfico generado en tiempo de ejecución; **nunca debe versionarse** (todos están en `.gitignore`).

Variable	Default	Función
FLASK_ENV	development	Selecciona la clase de configuración.
SECRET_KEY	<i>inseguro</i>	Firma de cookies de sesión (debe cambiarse en prod).
ENCRYPTION_KEY_PATH	key.key	Ruta de la llave Fernet activa.
ENCRYPTION_LEGACY_KEY_PATHS	(vacío)	Llaves “legadas” para descifrar durante rotación.
CERT_CA_CERT_PATH	certs/ca_cert.pem	Certificado de la CA local.
CERT_CA_KEY_PATH	certs/ca_key.pem	Llave privada de la CA local.
LOGIN_MAX_ATTEMPTS	5	Intentos antes de bloquear.
LOGIN_WINDOW_SECONDS	300	Ventana de conteo de intentos.
LOGIN_LOCKOUT_SECONDS	900	Duración del bloqueo.
PASSWORD_MIN_LENGTH	12	Longitud mínima de contraseña.
SIGNATURE_CHALLENGE_TTL_SECONDS	300	Vigencia del reto de firma.
PASSKEY_ENABLED	1	Habilita WebAuthn.
PASSKEY_ENFORCE_CRITICAL	0	Exige passkey a roles críticos.
PASSKEY_RP_ID / PASSKEY_ORIGIN	localhost / http://localhost:5000	Identidad de la <i>Relying Party</i> .
ENABLE_SESSION_COOKIE_SECURE	1 (prod)	Fuerza cookies Secure.

Cuadro 5.1: Variables de entorno principales.

Capítulo 6

Instalación y Ejecución

6.1. Pasos para levantar el sistema

1. Clonar el repositorio y entrar al directorio del proyecto.
2. Ejecutar el *script* de configuración: `./setup.sh`. Este crea el entorno virtual `.venv`, instala dependencias, copia `.env.example` a `.env`, genera `key.key` y crea las carpetas `certs/`, `backups/` y `logs/`.
3. Editar `.env` con los valores apropiados (sobre todo `SECRET_KEY` en producción).
4. Ejecutar la aplicación: `.venv/bin/python app.py`. En el primer arranque, `init_db()` crea/migra el esquema, `create_default_accounts()` siembra cuentas por defecto y `bootstrap_dev_certificates()` genera la CA y certificados de desarrollo.
5. Abrir `http://127.0.0.1:5000` en el navegador.

6.2. Comandos principales

```
# Configuración inicial (entorno, deps, .env, key.key, carpetas)
./setup.sh

# Ejecutar la app (sirve en http://127.0.0.1:5000)
.venv/bin/python app.py

# Verificación de sintaxis
.venv/bin/python -m py_compile app.py

# Suite de pruebas completa
PYTHONPATH=. .venv/bin/pytest -q

# Una prueba puntual
PYTHONPATH=. .venv/bin/pytest tests/test_password_security.py -q

# Generar/regenerar la llave Fernet de cifrado de datos
.venv/bin/python generate_key.py

# Respaldo / restauración cifrados de la base de datos
.venv/bin/python tools/backup_db.py
.venv/bin/python tools/restore_db.py
```

No existe un comando de *lint/format* dedicado: la vía de verificación establecida es `py_compile` más `pytest`.

Capítulo 7

Base de Datos

7.1. Modelo de datos

La persistencia es **SQLite** (`database.db`), accedida con `sqlite3` crudo a través de `get_conn()` (con `row_factory` para filas tipo diccionario). El esquema se crea y evoluciona en `init_db()` usando el patrón `ensure_column(conn, tabla, columna, definición)`, que añade columnas de forma **idempotente** sobre bases de datos existentes —evitando migraciones destructivas. Consta de **11 tablas**.

7.2. Tablas principales

Tabla	Contenido
usuarios	Cuentas: <code>username</code> , <code>password_hash</code> (Argon2id), <code>role</code> , <code>must_change_password</code> , estado de identidad reforzada.
encuestas	Expedientes: <code>estado</code> , <code>datos</code> (<i>blob</i> cifrado Fernet), <code>encryption_key_fingerprint</code> , metadatos de revisión.
logs	Bitácora: <code>usuario</code> , <code>accion</code> , <code>categoria</code> (operacion/seguridad), <code>detalle</code> , <code>timestamp</code> , <code>ip_address</code> .
encryption_keys	Ciclo de vida de llaves: <code>key_fingerprint</code> (SHA-256), <code>state</code> (activo/legada/retiring).
encryption_metrics	Métricas de latencia/uso de cifrado y descifrado.
reencrypt_jobs	Cola de re-cifrado masivo durante una rotación de llave.
solicitudes_eliminacion	Solicitudes de eliminación de expedientes y su resolución.
login_lockouts	Bloqueo de cuentas: <code>intentos</code> , <code>ventana</code> , <code>locked_until</code> .
certificados	Certificados X.509: <code>serial_number</code> , <code>certificate_pem</code> , <code>estado</code> , <code>expires_at</code> , <code>algorithm</code> .
passkey_credentials	Credenciales WebAuthn: <code>credential_id</code> , <code>public_key</code> (COSE), <code>sign_count</code> , <code>cert_id</code> .
solicitudes_arco	Solicitudes ARCO con aprobación por nivel y <code>nivel_actual</code> .

Cuadro 7.1: Tablas de la base de datos.

7.3. Relaciones

Las relaciones se modelan de forma ligera mediante claves foráneas lógicas (SQLite no exige su declaración, pero el código las respeta):

- `encuestas.usuario_id` → `usuarios.id` (autor del expediente; uno a muchos: un usuario crea muchos expedientes).
- `certificados.user_id` → `usuarios.id` (uno a muchos histórico, pero solo un certificado activo por usuario).
- `passkey_credentials.user_id` → `usuarios.id` y `passkey_credentials.cert_id` → `certificados.id`. Esta última es la relación **N:1** clave: varias passkeys de un usuario se enlazan a un mismo certificado derivado.
- `logs.usuario` referencia (por nombre) a `usuarios.username` para fines de auditoría.
- `encuestas.encryption_key_fingerprint` corresponde a un `encryption_keys.key_fingerprint`, indicando con qué llave se cifró el registro.

Capítulo 8

API / Interfaces

La aplicación **no expone una API JSON/REST de propósito general**: cada ruta renderiza una plantilla Jinja2 o redirige tras procesar un formulario. Las respuestas JSON se limitan a los *endpoints* WebAuthn (que siguen el protocolo del navegador) y a algunos paneles de administración consumidos por `fetch()`. Por ello, en este capítulo “API/Interfaces” se documentan las **rutas HTTP** como interfaz del sistema.

8.1. Endpoints

La Tabla 8.1 resume las rutas por área funcional (40 rutas en total).

8.2. Métodos y parámetros

Las rutas siguen la convención HTTP estándar: GET para vistas y POST para operaciones que modifican estado. Los parámetros se reciben como **campos de formulario** (`request.form`), **parámetros de ruta** (p. ej. `<int:encuesta_id>`) o, en los *endpoints* WebAuthn, como **cuerpo JSON**. Toda solicitud que modifica estado debe incluir el **token CSRF** de la sesión (ver el capítulo de Manejo de Errores y Seguridad).

8.3. Ejemplos de uso

Avanzar un expediente (formulario HTML que dispara POST `/encuesta/<id>/avanzar`):

```
<form method="POST" action="/encuesta/42/avanzar">
  <input type="hidden" name="csrf_token" value="{ csrf_token }">
  <button type="submit">Avanzar al siguiente estado</button>
</form>
```

Iniciar el registro de una passkey (cliente, JSON):

```
const resp = await fetch('/auth/passkey/register/options', {
  method: 'POST',
  headers: {'Content-Type': 'application/json', 'X-CSRF-Token': csrfToken}
```

Ruta	Métodos	Función
<i>Autenticación e identidad</i>		
/	GET, POST	Login (Argon2id, bloqueo, gating de identidad).
/auth/passkey/register/options verify	POST	Registro WebAuthn.
/auth/passkey/login/options verify	POST	Autenticación WebAuthn.
/auth/check-passkey-required	POST	Indica si la cuenta exige passkey.
/action/passkey/options verify	POST	Firma de acciones críticas con passkey.
/certificado/setup	GET, POST	Alta de certificado X.509 (servidor o CSR).
/password/update	GET, POST	Cambio de contraseña.
/logout	GET	Cierre de sesión.
<i>Expedientes</i>		
/dashboard	GET	Panel según rol.
/survey	GET, POST	Alta de expediente (cifra datos).
/bandeja	GET	Pendientes de revisión del rol.
/encuesta/<id>/avanzar	POST	Avanza el estado del expediente.
/solicitar-eliminacion/<id>	POST	Solicita eliminación.
/solicitud/<id>/resolver	POST	Resuelve la solicitud.
<i>Usuarios, PKI y passkeys</i>		
/usuarios,/eliminar-usuario/<id>	GET/POST	Gestión de cuentas.
/admin,/admin/identidades,/admin/pki	GET	Paneles de administración.
/admin/pki/revoke-passkey revoke-certificate	POST	Revocaciones.
/certificado/<id>/descargar revocar	POST	Descarga/revoca certificado.
<i>Cifrado y rotación</i>		
/admin/cifrado,/admin/cifrado/metrics jobs	GET/POST	Estado, métricas y trabajos.
/admin/keys/configure	POST	Configura llave candidata (firma passkey).
/admin/keys/<fp>/activate	POST	Activa llave y encola re-cifrado.
<i>Auditoría, perfil y ARCO</i>		
/logs,/logs/clear	GET/POST	Bitácora y limpieza (solo admin).
/profile	GET, POST	Perfil del usuario.
/arco,/arco/solicitud	GET/POST	Formulario y alta de solicitud.
/arco/<id>/aprobar resolver resolver-coordinador	POST	Pipeline multinivel.

Cuadro 8.1: Rutas HTTP (interfaz del sistema).

```
});
const options = await resp.json();
const credential = await navigator.credentials.create({ publicKey: options });
// ... se envia el atestado a /auth/passkey/register/verify
```

Capítulo 9

Flujos y Lógica de Negocio

9.1. Procesos clave del sistema

Inicio de sesión y refuerzo de identidad

El login (/) valida usuario/contraseña con `verify_password_argon2id`. Antes de comprobar la contraseña, `_is_locked()` verifica si la cuenta/IP está bloqueada. Tras un fallo, `_record_failed()` incrementa el contador; al alcanzar el máximo se fija el bloqueo y se registra en logs con `categoria="seguridad"`. Para roles críticos, `enforce_certificate_setup()` (*hook before_request*) obliga a completar y usar certificado/passkey antes de operar.

Ciclo de vida del expediente

La máquina de estados (`next_status_for_role()`) gobierna las transiciones, atando cada una al rol que corresponde revisar ese estado:

```
borrador → en_revision_operativa → en_revision_coordinacion → validado_coordinacion →  
                                              cerrado
```

Cifrado y rotación de llaves

`encrypt_data()` cifra con la llave activa y antepone la huella de la llave; el formato del *blob* resultante es:

```
b"fp:" + <huella SHA-256 de la llave> + b"\n" + <token Fernet>
```

`decrypt_data()` lee la huella, ubica la llave en el *keyring* (`get_cipher_for_fingerprint`) —que incluye la activa y las legadas— y descifra. La rotación configura/activa una nueva llave y encola `reencrypt_jobs`, que `tools/reencrypt_worker.py` procesa en lotes.

Solicitudes ARCO multinivel

/arco → usuarios → operativos → coordinadores → {resuelto | reenviado a admin}

Cada nivel registra su aprobación en columnas dedicadas (aprobado_<nivel>, _at, _por).

9.2. Reglas importantes

- Una transición de estado solo procede si el expediente está en el estado que el rol actual puede mover; de lo contrario se rechaza.
- Las acciones críticas (rotar llave, revocar identidades) exigen firma con passkey vía `check_and_consume_passkey_action` (vigencia de 60 s).
- Los retos de firma (certificado) son de un solo uso y caducan a los 300 s.
- Si un certificado expira, `check_certificate_expiration` desactiva automáticamente las passkeys vinculadas (relación N:1).
- Toda contraseña nueva pasa por `validate_new_password_policy` (longitud, entropía y verificación HIBP).

9.3. Casos críticos

- **Cuenta bloqueada:** tras 5 intentos fallidos en 5 minutos, la cuenta queda bloqueada 15 minutos; el contador se reinicia al expirar la ventana.
- **Llave activa perdida sin rotación ordenada:** sustituir `key.key` sin el flujo de rotación deja ilegibles los datos cifrados con la llave anterior.
- **Certificado comprometido:** debe *revocarse* de inmediato (no basta esperar a su expiración), lo que además desactiva sus passkeys.

Capítulo 10

Ejemplos de Uso

10.1. Casos típicos de uso

Crear y canalizar un expediente. Un usuario entra a `/survey`, llena el formulario y lo guarda (estado borrador, datos cifrados). El operativo lo ve en su `/bandeja` y lo avanza; luego el coordinador lo revisa y valida; finalmente el admin lo cierra. Cada paso queda en la bitácora.

10.2. Integración básica

Aunque no hay SDK/cliente externo, los *helpers* criptográficos pueden usarse de forma programática dentro del proyecto. Por ejemplo, para cifrar y descifrar un dato sensible:

```
from app import encrypt_data, decrypt_data

blob = encrypt_data("Informacion sensible del expediente")
# blob == b"fp:<huella>\n<token Fernet>"

texto = decrypt_data(blob) # usa el keyring (llave activa + legadas)
assert texto == "Informacion sensible del expediente"
```

10.3. Ejecución de funciones clave

Hashing y verificación de contraseña (Argon2id):

```
from app import hash_password_argon2id, verify_password_argon2id

hashed = hash_password_argon2id("UnaContrasenaFuerte!2026")
ok = verify_password_argon2id("UnaContrasenaFuerte!2026", hashed) # True
```

Generación de la llave Fernet (desde la línea de comandos):

```
.venv/bin/python generate_key.py # escribe key.key
```

Capítulo 11

Manejo de Errores

11.1. Estrategia de errores

El sistema no usa un *framework* de errores estructurado tipo JSON. Los errores de negocio se comunican con **mensajes flash** renderizados en las plantillas (`flash()` + `render_template()`); los flujos WebAuthn/AJAX responden **JSON** con un código HTTP apropiado y un campo de error legible. Los fallos criptográficos (firma inválida, certificado expirado, reto vencido, descifrado fallido) producen un mensaje específico al usuario y, en paralelo, una entrada de auditoría con `categoria="seguridad"`.

La protección **CSRF** es global (*hook before_request*): `ensure_csrf_token()` genera el token, `validate_csrf()` lo compara y `csrf_protect()` rechaza toda solicitud que modifique estado sin un token válido.

11.2. Mensajes comunes

- “Credenciales incorrectas.” — usuario o contraseña inválidos.
- “Cuenta bloqueada temporalmente.” — se superó `LOGIN_MAX_ATTEMPTS`.
- “Debe configurar su certificado/passkey.” — *gating* de identidad para rol crítico.
- “Firma inválida” / “Reto expirado.” — fallo en reto–respuesta.
- “Contraseña insegura o presente en filtraciones.” — rechazo de política/HIBP.
- “Token CSRF inválido.” — formulario sin token o token incorrecto.

11.3. Registro (logs)

La función `log(usuario, accion, categoria="operacion", detalle=None)` inserta en la tabla `logs` con marca de tiempo e IP. Se invoca en cada operación sensible: logins (éxito/fallo/bloqueo), cambios de estado, emisión/revocación de certificados y passkeys, configuración/rotación de llaves, resoluciones ARCO y eliminación de usuarios. La categoría `seguridad` separa los eventos de interés forense de las operaciones de negocio (`operacion`). La ruta `/logs` permite visualizar la bitácora; `/logs/clear` su limpieza (solo `admin`).

Capítulo 12

Pruebas

12.1. Tipos de tests

La suite vive en `tests/` y usa *fixtures* (`app_module`) que aplican `monkeypatch.chdir(tmp_path)` para reconstruir, por cada prueba, una base de datos, un conjunto de certificados y llaves **aislados**. La cobertura incluye:

- Seguridad de contraseñas: rechazo de contraseñas débiles y bloqueo de hashes “legados”.
- Límite de intentos de login y bloqueo temporal.
- Emisión, activación, revocación y flujo CSR de certificados X.509.
- Firma de acciones críticas con passkeys y verificación CSRF.
- Ciclo completo de canalización de un expediente entre los cuatro roles.
- Flujo y escalamiento de solicitudes ARCO multinivel (incluido el reenvío a admin).

Los archivos `test_arco_*.py`, `test_csrf_arco.py` y `test_final_verification.py` en la raíz son *scripts* de verificación ad hoc, no parte de la suite `pytest` formal.

12.2. Cómo ejecutarlos

```
# Suite completa
PYTHONPATH=. .venv/bin/pytest -q

# Un archivo
PYTHONPATH=. .venv/bin/pytest tests/test_password_security.py -q

# Una funcion especifica
PYTHONPATH=. .venv/bin/pytest \
    tests/test_password_security.py::test_qa_a_rejects_weak_password_on_forced_update -q
```

Capítulo 13

Despliegue

13.1. Cómo hacerlo

Despliegue local / autoalojado (referencia)

1. Ejecutar `setup.sh` en el servidor de destino.
2. Ajustar `.env` para producción: `FLASK_ENV=production`, `SECRET_KEY` fuerte y única, `ENABLE_SESSION_COOKIE_SECURE=1`.
3. Ejecutar la app detrás de un *proxy* reverso/terminador TLS (la app no incluye servidor WSGI de producción ni *proxy* propios).
4. Resguardar `key.key`, `certs/*.pem` y `database.db` con permisos restrictivos.
5. Programar respaldos cifrados periódicos con `tools/backup_db.py` y validar la restauración.

Despliegue en HostGator (hosting de producción)

En el hosting compartido de **HostGator**, la aplicación se publica como aplicación Python a través de *Passenger*/cPanel (“Setup Python App”):

1. Crear la aplicación Python en cPanel, seleccionando la versión de Python (3.10+ si está disponible) y el directorio del proyecto.
2. Instalar dependencias en el entorno virtual que cPanel administra (`pip install -r requirements.txt`).
3. Definir las variables de entorno desde el panel (las de la Tabla 5.1), en especial `SECRET_KEY`, `FLASK_ENV=production` y los `PASSKEY_*` con el dominio real (`PASSKEY_RP_ID` y `PASSKEY_ORIGIN` deben coincidir con el dominio HTTPS de producción, requisito de WebAuthn).
4. Subir el material de llaves (`key.key`, CA) por un canal seguro y fuera de la raíz pública.
5. Apuntar el `passenger_wsgi` al objeto `app` de Flask.

13.2. Entornos

Se distinguen tres entornos seleccionados por `FLASK_ENV`:

- **development:** `DEBUG=True`, cookies sin `Secure`; para trabajo local.
- **production:** `DEBUG=False`, `SameSite=Strict`, cookies `Secure`; para HostGator.
- **testing:** *rate-limiting* desactivado; para la suite de pruebas.

13.3. Consideraciones

- **HTTPS obligatorio para WebAuthn:** las passkeys solo funcionan sobre un origen seguro; el dominio de HostGator debe servirse por HTTPS y coincidir con `PASSKEY_ORIGIN`.
- **SQLite en hosting compartido:** el bloqueo de archivos puede degradarse bajo concurrencia; conviene mantener un solo proceso/worker.
- **Procesos en segundo plano:** el *worker* de re-cifrado (`reencrypt_worker.py`) puede requerir ejecución por *cron* si el hosting no permite procesos de larga duración (ver Problemas Conocidos).
- **Secretos fuera del repositorio:** nunca subir `key.key`, `certs/*.pem` ni `.env` al control de versiones ni a directorios públicos.

Capítulo 14

Seguridad

14.1. Autenticación

La autenticación base usa **Argon2id** (`Type.ID`) con `memory_cost=65536` (64 MiB), `time_cost=3`, `parallelism=2`, `hash_len=32` y una sal aleatoria de 16 bytes generada con `os.urandom`. Argon2id es *memory-hard* y resistente tanto a ataques de canal lateral como a fuerza bruta con GPU/ASIC. Para roles críticos se añade un segundo factor de **posesión**: certificado X.509 con reto–respuesta (RSA, PKCS1v15 + SHA256) y/o passkey WebAuthn. Las passkeys son **resistentes a phishing** porque la firma queda atada al `PASSKEY_ORIGIN`, y verifican el `sign_count` para detectar clonación de autenticadores.

14.2. Manejo de datos sensibles

Los datos sensibles de cada expediente se cifran a nivel de campo con **Fernet** (AES-128-CBC + HMAC-SHA256, *cifrado autenticado*). Cada *blob* lleva la **huella SHA-256** de la llave usada, lo que permite **rotación de llaves** sin perder acceso a datos antiguos: conviven una llave activa y llaves legadas en el *keyring*. La rotación sigue el ciclo activo → legada → retirada, con re-cifrado masivo en segundo plano. Los respaldos de la base de datos también se cifran con Fernet.

14.3. Buenas prácticas

- **Verificación de contraseñas filtradas** con HIBP por k-anonimato: solo se envían los 5 primeros caracteres hex del SHA-1 de la contraseña; la comparación del sufijo es local.
- **Cookies endurecidas** (`enforce_cookie_flags`): `HttpOnly`, `Secure` y `SameSite`.
- **CSRF** global con token sincronizado por sesión.
- **Mínimo privilegio** y **separación de funciones** vía RBAC y la máquina de estados de expedientes.
- **Secretos fuera del repositorio** (todo el material criptográfico está en `.gitignore`).
- **Auditoría** de eventos de seguridad para no repudio y rendición de cuentas.

Capítulo 15

Guía de Contribución

15.1. Convenciones de código

- Todo el backend reside en `app.py`; al agregar lógica, ubíquela cerca de las funciones afines y respete el estilo existente (nombres en español para rutas, columnas y mensajes; funciones *snake_case*).
- Para evolucionar el esquema, use **siempre** `ensure_column()` (idempotente) en lugar de `ALTER TABLE` directo o migraciones destructivas.
- No imprima ni registre material secreto (llaves, PEM, contraseñas).
- Verifique con `py_compile` y ejecute `pytest` antes de proponer cambios.

15.2. Flujo de trabajo (git)

1. Crear una rama a partir de `main` (no trabajar directamente sobre `main`).
2. Realizar *commits* pequeños y descriptivos (en español, consistente con el historial).
3. Abrir un *Pull Request* describiendo qué cambia, por qué y cómo se probó.
4. No incluir en el control de versiones `key.key`, `certs/*.pem`, `keys/`, `.env` ni `*.db`.

15.3. Cómo agregar funcionalidades

1. **Nueva ruta:** añadir un `@app.route` con el decorador `require_role()` apropiado y validar permisos con `has_permission()`.
2. **Nueva tabla/columna:** declararla en `init_db()` y, si aplica a bases existentes, añadirla con `ensure_column()`.
3. **Operación crítica:** protegerla con `check_and_consume_passkey_action` y registrar el resultado con `log(..., categoria="seguridad")`.
4. **Plantilla:** crear el HTML correspondiente en `templates/` incluyendo el campo `csrf_token` en todo formulario POST.
5. **Pruebas:** agregar casos en `tests/` siguiendo el patrón de *fixtures* aislados.

Capítulo 16

Problemas Conocidos

16.1. Bugs actuales

Al tratarse de la primera versión estable del gestor, no hay un registro de *bugs* abiertos considerados bloqueantes. Las áreas a vigilar son la concurrencia sobre SQLite y la coordinación del *worker* de re-cifrado durante rotaciones de llave.

16.2. Limitaciones

Las principales limitaciones provienen del modelo de despliegue en **hosting compartido (HostGator)** y de la arquitectura de archivo único:

- **Soporte de Python/WSGI restringido:** la versión de Python y las extensiones nativas disponibles dependen del plan; `cryptography` y `argon2-cffi` requieren ruedas compatibles. Conviene verificar la versión antes de desplegar.
- **Sin procesos de larga duración:** el hosting compartido suele matar procesos persistentes, por lo que el *worker* de re-cifrado debe ejecutarse de forma programada (cron) en lotes, en vez de como demonio continuo.
- **Concurrencia limitada con SQLite:** el bloqueo a nivel de archivo y los límites de E/S del hosting compartido restringen el número de escrituras simultáneas; el sistema se piensa para un solo proceso.
- **Límites de recursos:** cuotas de CPU, memoria y conexiones del plan compartido pueden afectar el costo de Argon2id (64 MiB por verificación) bajo picos de login.
- **Control parcial del entorno:** la terminación TLS, las cabeceras y el *cron* los administra el panel del proveedor, no la aplicación; se depende de su configuración para garantizar HTTPS (indispensable para WebAuthn).
- **Sin escalado horizontal:** no hay estado compartido entre instancias; replicar el proceso requeriría migrar de SQLite a un motor cliente-servidor.

Capítulo 17

FAQ Técnica

17.1. Problemas comunes al levantar el proyecto

`ModuleNotFoundError` al ejecutar `app.py`.

Activar el entorno virtual (`source .venv/bin/activate` o usar `.venv/bin/python`) y reinstalar dependencias con `pip install -r requirements.txt` (o `setup.sh`).

La app no encuentra `key.key`.

Generarla con `python generate_key.py` o dejar que `setup.sh` la cree. En HostGator, subirla fuera de la raíz pública.

Las passkeys fallan en producción.

Verificar que `PASSKEY_RP_ID` y `PASSKEY_ORIGIN` coincidan con el dominio HTTPS real; WebAuthn exige origen seguro.

Cuenta bloqueada en pruebas manuales.

Esperar `LOGIN_LOCKOUT_SECONDS` (15 min) o usar el entorno `testing`, que relaja el *rate-limiting*.

17.2. Errores típicos de desarrollo

Datos ilegibles tras regenerar la llave.

Reemplazar `key.key` sin pasar por el flujo de rotación (`/admin/keys/configure + /admin/keys/<fp>/activate + worker`) deja ilegibles los registros previos. Conservar la llave anterior como “legada” hasta completar el re-cifrado.

“Token CSRF inválido” en formularios nuevos.

Olvidar incluir `{{ csrf_token }}` en un formulario POST.

Cambios de esquema que rompen bases existentes.

Usar `ALTER TABLE` directo en lugar de `ensure_column()`.

Permisos insuficientes en una ruta nueva.

Olvidar `require_role()/has_permission()`, dejando la acción accesible a roles que no deberían ejecutarla.

Capítulo 18

Referencias

- Documentación de cryptography (Fernet, X.509, RSA): <https://cryptography.io/>
- argon2-cffi (Argon2id): <https://argon2-cffi.readthedocs.io/>
- Biblioteca webauthn (py_webauthn): https://github.com/duo-labs/py_webauthn
- Especificación WebAuthn (W3C): <https://www.w3.org/TR/webauthn-2/>
- Have I Been Pwned — API de contraseñas (k-anonimato): <https://haveibeenpwned.com/API/v3#PwnedPasswords>
- RFC 9106 — Argon2: <https://www.rfc-editor.org/rfc/rfc9106>
- NIST SP 800-57 — Recomendaciones de gestión de llaves: <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>
- Documentación de Flask: <https://flask.palletsprojects.com/>
- Configuración de aplicaciones Python en HostGator/cPanel (Setup Python App): <https://www.hostgator.com/help>